

тельный процессор. АПП найдет широкое применение в различных областях науки и техники, где требуется обработка больших массивов информации, в таких как: системы обработки сигналов, слежения и управления большим количеством однотипных процессов или объектов управления уличным движением, анализа треков взаимодействия ядерных частиц в ядерных исследованиях и др. Построенный на основе микропроцессорных интегральных схем АПП может легко адаптироваться к задачам конкретного пользователя, заняв тем самым достойное место среди устройств обработки данных.

СПИСОК ЛИТЕРАТУРЫ

1. А. С. 122639 (СССР). — Оpubл. в БИ, 1959, № 18.
2. Slade A. E., ... A cryotron ctalog memory sustem. Proc. of East. Joint. Computer Conference, 1956.
3. Аксенов В. П. Ассоциативные процессоры и области их применения. — Зарубежная радиоэлектроника, 1977, № 1.
4. Мультипроцессорные системы и параллельные вычисления: Пер. с англ./ Под ред. Ф. Г. Энслоу. — М.: Мир, 1976.
5. Однородные микрoэлектронные ассоциативные процессоры: Пер. с англ./ Под ред. И. В. Прангшвили. — М.: Сов. радио, 1973.
6. Слободянюк Т. Ф., Яковлев Ю. С. Реализация арифметических и логических операций в запоминающем устройстве. — Управляющие машины и системы/ Изд-во АН УССР, 1968, вып. 4.
7. Попова Г. М., Прангшвили И. В. Ассоциативный параллельный процессор для групповой обработки данных. — Автоматика и телемеханика, 1972, № 1.
8. Баумс А. К., Гуртовцев А. Л., Зазнова Н. Е. Микропроцессорные средства. — Рига: Зинатне, 1977.
9. Макаревич О. Б., Спиридонов Б. Г., Кравченко П. П. Микро-ЭВМ как средство обработки информации. — Зарубежная электронная техника, 1977, № 9—11.

УДК 681.3.06

А. В. КОБЫЛИНСКИЙ, Н. Г. САБАДАШ, А. К. ТЕСЛЕНКО

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ МИКРО-ЭВМ НА БАЗЕ ОДНОКРИСТАЛЬНОГО МИКРОПРОЦЕССОРА

Одним из условий успешного внедрения микропроцессорных БИС является наличие эффективного программного обеспечения (ПО) для микро-ЭВМ, построенных на их основе. Поскольку микро-ЭВМ по организации вычислительного процесса мало отличаются от уни-

версальных ЭВМ (УЭВМ), то при разработке ПО микро-ЭВМ можно использовать методы и идеи, положенные в основу разработки ПО УЭВМ. Однако при этом необходимо учитывать специфику структуры, реализации и областей применения микро-ЭВМ. Основной областью применения микро-ЭВМ на современном этапе является управление в реальном масштабе времени различными объектами и системами. Важной характери-

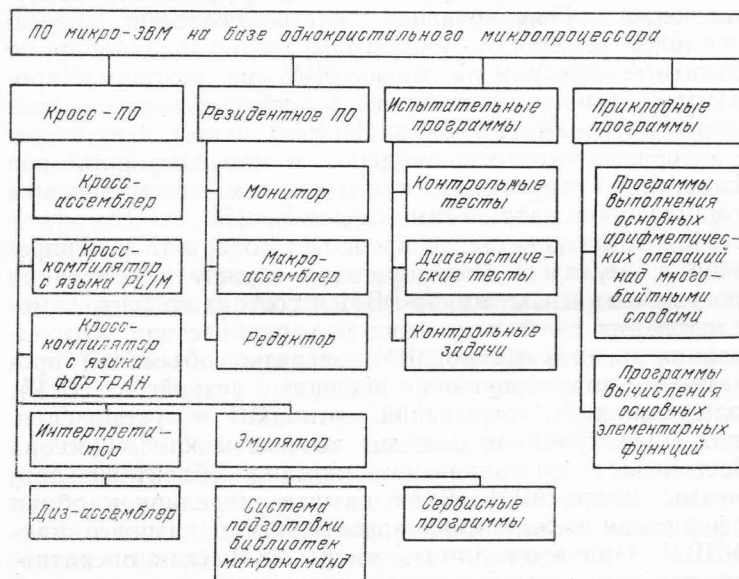


Рис. 1. Структура ПО микро-ЭВМ на базе однокристалльного микропроцессора

стикой микро-ЭВМ для таких систем является минимальная стоимость как аппаратуры, так и подготовки и отладки программ, что вызывает противоречивые требования минимизации объема ЗУ микро-ЭВМ и наличия эффективной системы подготовки и отладки программ. Можно указать на следующие два пути решения этой задачи:

— разработка системы подготовки и отладки программ микро-ЭВМ на базе УЭВМ (кросс-СПО);

— разработка достаточно эффективной системы подготовки и отладки программ на базе микро-ЭВМ (ре-

зидентная СПО) при ограниченном объеме используемого ЗУ.

В работе рассматривается кросс-СПО и резидентная СПО для микро-ЭВМ на базе однокристалльного микропроцессора. На рис. 1 изображен состав ПО микро-ЭВМ. **Кросс-ПО** представляет собой комплекс программ УЭВМ, написанных на языке программирования высокого уровня, например на языке ФОРТРАН, и состоит из трех частей: генерирующей, моделирующей и обслуживающей. Генерирующая часть содержит кросс-ассемблер и кросс-компиляторы, которые генерируют объектные программы микро-ЭВМ из исходных программ, составленных на языке ассемблера или на языках высокого уровня (*PL/M*, ФОРТРАН и др.). Структуры и алгоритмы кросс-ассемблеров и кросс-компиляторов аналогичны структурам и алгоритмам ассемблеров и компиляторов, разработанным для УЭВМ.

Моделирующая часть кросс-ПО содержит интерпретатор и эмулятор. Интерпретатор служит для отладки программ на языке микро-ЭВМ и состоит из блоков моделирования системы команд микропроцессора, моделирования памяти микро-ЭВМ, загрузки объектной программы, моделирования внешних устройств (ВУ), моделирования прерываний, отладки и управления. Блок моделирования системы команд микропроцессора обеспечивает интерпретацию команд объектной программы микро-ЭВМ. Блок памяти моделирует обмен информации между микропроцессором и памятью микро-ЭВМ. При этом память микро-ЭВМ (как оперативная, так и постоянная) моделируется полями оперативной памяти УЭВМ. Блок загрузки предназначен для размещения объектной программы микро-ЭВМ в соответствующей области оперативной памяти УЭВМ. Блок ВУ обеспечивает моделирование ВУ микро-ЭВМ с помощью ВУ УЭВМ.

Примерный состав моделируемых операций следующий: ввод — вывод байтов на перфоленту, на магнитную ленту (МЛ), на магнитный барабан или диск (МБ или МД), на перфокарты в шестнадцатиричном, десятичном, восьмеричном или текстовом формате и т. д. Блок моделирования прерываний служит для формирования моделей сигналов запроса прерывания, используя, например, генератор псевдослучайных чисел. Блок отладки предназначен для реализации команд трассировки,

разгрузки и событий, используемых при отладке программ. Блок управления организует работу перечисленных блоков и реализует команды управления интерпретатора.

Эмулятор, входящий в состав моделирующей части кросс-ПО, предназначен для отладки исходных программ микро-ЭВМ, составленных на языке ассемблера или языке высокого уровня. Эмулятор организует трансляцию исходной программы при помощи кросс-ассемблера или кросс-компилятора с последующим просчетом объектной программы при помощи интерпретатора.

Обслуживающая часть кросс-ПО содержит диз-ассемблер, систему организации библиотеки макрокоманд и ряд сервисных программ. Диз-ассемблер предназначен для формирования и выдачи листингов программ на языке ассемблера по программам на машинном языке микро-ЭВМ. Сервисные программы обеспечивают организацию программных файлов, выдачу исходных и объектных программ на перфоленте для последующего ввода в микро-ЭВМ, подготовку документации по отлаженным программам — распечатки таблиц прошивок и карт заказа постоянных запоминающих устройств (ПЗУ), листингов программ, протоколов отладки и т. д.

Резидентное ПО микро-ЭВМ — это ПО, реализуемое на микро-ЭВМ непосредственно. В минимальный состав резидентного ПО входит управляющая программа-монитор, макроассемблер и редактор программ. Монитор предназначен для загрузки, управления выполнением и отладки программ пользователя и записывается в ПЗУ. В состав монитора входят блоки задания конфигурации микро-ЭВМ, загрузки, управления выполнением программ пользователя, драйверов, отладки и сервисный блок.

Блок задания конфигурации микро-ЭВМ предназначен для задания соответствия логическим ВУ, выделенным по функциональному признаку (например, устройство отображения выводимой информации, перфоратор, пульт и др.), физическим ВУ, присущим данной конфигурации микро-ЭВМ. Так, логическому устройству пульта может соответствовать телетайп, дисплей или электрическая пишущая машинка (ЭПМ); логическому устройству отображения выводимой информации — также телетайп, дисплей или ЭПМ; логическому устройству перфорации — телетайп или перфоратор и т. д. Блок

загрузки предназначен для считывания программ пользователя с перфоленты и размещения их в требуемую область оперативной памяти микро-ЭВМ. Блок драйверов содержит драйверы ВУ микро-ЭВМ и обеспечивает обмен информацией между микропроцессором и ВУ. Блок отладки предназначен для выполнения команд разгрузки и трассировки, а также для модификации содержимого памяти и регистров микропроцессора. Сервисный блок позволяет вводить перфоленту с одновременной распечаткой ее содержимого, выводить на перфоленту отлаженные программы, выполнять пересылку из одной области оперативной памяти в другую и др.

Структура и состав макроассемблера микро-ЭВМ в основном аналогичны структуре и составу макроассемблеров УЭВМ. Резидентный макроассемблер является двухпроходным со встроенным макрогенератором. Язык ассемблера микро-ЭВМ характеризуется следующими особенностями. Отсутствует жесткая привязка полей команд языка. В выражениях наряду с арифметическими операциями (сложение, вычитание, умножение, деление) допускаются поразрядные логические операции и операции сдвига вправо и влево. Введены псевдокоманды, позволяющие отменить трансляцию заданных участков исходных программ по выполнению некоторого условия (условное ассемблирование), повторно присваивать символическим именам абсолютные значения. Макрокоманды языка могут содержать до 126 фактических параметров. Параметры в макрокомандах являются позиционными. В зависимости от заданного признака в макрорасширении подставляются значения фактических параметров или сами параметры. Такое различие стало возможным благодаря введению псевдокоманды, переопределяющей абсолютные значения символических имен. В теле макроопределения допускаются глобальные метки, т. е. метки, к которым можно обращаться с любых участков программы. Для устранения дублирования глобальных меток при многократных использованиях таких макрокоманд применяется условное ассемблирование.

Перечисленные особенности языка ассемблера микро-ЭВМ позволяют повысить эффективность исходных программ, облегчить программирование и отладку.

Редактор программ, входящий в состав резидентного ПО микро-ЭВМ, предназначен для автоматизации под-

готовки и исправления исходных программ на языке ассемблера. Редактор содержит блоки ввода, редактирования и вывода. Блок ввода позволяет формировать текстовый файл в рабочей области редактора путем записи (дозаписи) символов, вводимых с пульта или устройства считывания с перфоленты или путем вставки символов, вводимых с пульта, в требуемое место текстового файла. Блок редактирования позволяет производить поиск требуемых символов в файле, а также исключать символы из файла. Блок вывода предназначен для распечатки или перфорации отредактированного файла.

Испытательные программы, входящие в состав ПО микро-ЭВМ, предназначены для контроля и диагностики неисправностей в аппаратуре микро-ЭВМ. В состав испытательных программ входят контрольные и диагностические тесты, контрольные задачи.

Пакет прикладных программ содержит подпрограммы (макрокоманды) выполнения основных арифметических операций над многобайтными словами и вычисления основных элементарных функций.

На основе рассмотренного ПО микро-ЭВМ организуется кросс-ПО и резидентная СПО.

Структура кросс-СПО изображена на рис. 2. Программы для микро-ЭВМ составляются на машинном языке, языке ассемблера или на одном из языков высокого уровня и наносятся на перфоленту или перфокарту. Программы на машинном языке кодируются в шестнадцатиричном или восьмеричном формате и считываются программой ввода, входящей в состав обслуживающей части кросс-ПО. Программа ввода формирует объектный файл и записывает его на внешний накопитель (ВН). МЛ, МБ или МД по адресу, задаваемому пользователем.

Кросс-ассемблер и кросс-компиляторы считывают исходную программу с перфоленты, перфокарты или из текстового файла на ВН, подготовленного редактором текстов, входящим в состав операционной системы УЭВМ. Таким образом развитые средства подготовки и редактирования текстовых файлов, присущие системному ПО УЭВМ, используются в кросс-ПО для подготовки и редактирования исходных программ микро-ЭВМ. В связи с этим разработка и включение в состав кросс-ПО специального редактора исходных программ не является целесообразной.

Кросс-ассемблер и кросс-компиляторы работают в стандартном или управляемом режимах. В стандартном режиме выдается листинг исходной программы. Стандартный режим служит для отладки исходных программ на уровне синтаксических ошибок.

Управляемый режим устанавливается с помощью команд, позволяющих отменить выдачу листинга, задать формирование объектного файла с последующей его за-

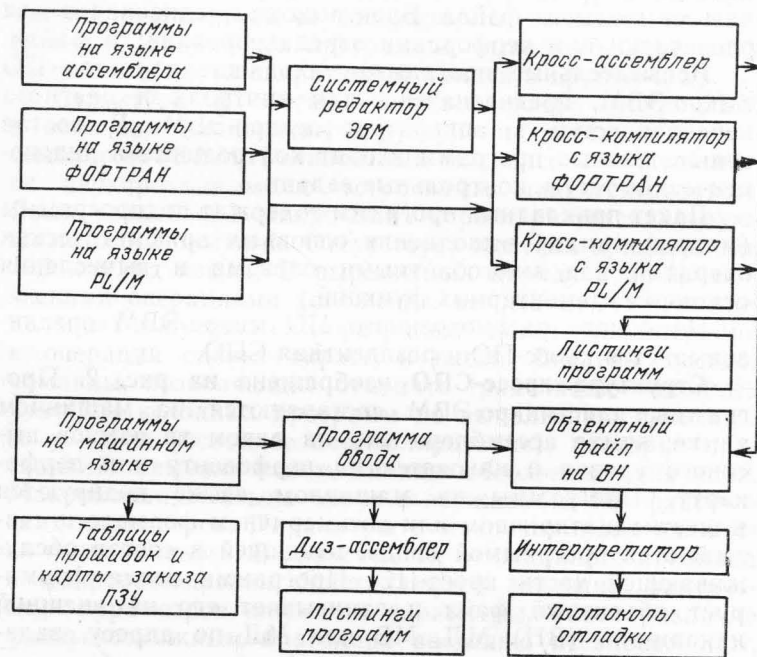


Рис. 2. Структура кросс-СПО

писью на ВН по адресу, указываемому пользователем, задать запись вслед за объектным файлом таблиц имен и меток, установить требуемый формат выдачи на листинг объектного кода в случае работы кросс-ассемблера и т. д. Кроме того, в случае работы кросс-ассемблера и использования в тексте исходной программы библиотечных макрокоманд указывается адрес на ВН библиотеки макрокоманд, подготовленной обслуживающей частью кросс-ПО.

Объектные программы, записанные на ВН, отлаживаются с помощью интерпретатора. Для организации работы интерпретатора предусматривается ряд команд, указывающих адрес объектного файла на ВН, начальный адрес программы, условий возникновения прерываний, областей памяти, отведенных под ПЗУ, формат распечатки строк трассировки, а также служащих для заказа ресурсов — времени счета в масштабе времени микро-ЭВМ и требуемых моделей ВУ микро-ЭВМ.

Собственно отладка производится путем задания команд трассировки, разгрузки и событий. Побуждением отладки является несоответствие реальной работы программы запланированной. В качестве такого несоответствия выступают неверные промежуточные или конечные результаты или же аварийные прерывания работы программы. Как правило, ошибка, следствием которой является обнаруженное несоответствие, удалена от него как во времени, так и по месту расположения в программе. В связи с этим условно отладку можно разделить на два этапа: первый (пассивный) — поиск несоответствий в работе программы или доказательство правильности программы, и второй (активный) — поиск ошибок по выявленным несоответствиям. Первому этапу соответствуют команды трассировки и разгрузки. Под трассировкой здесь подразумевается распечатка на АЦПУ УЭВМ строк трассировки после интерпретации команд микро-ЭВМ. Строка трассировки включает мнемокод или цифровой код команды, время, в масштабе времени микро-ЭВМ, прошедшее от начала выполнения программы до интерпретации текущей команды включительно, содержимое аккумулятора и регистров микропроцессора. Предусмотрена команда, при задании которой строка трассировки дополняется содержимым нескольких требуемых ячеек оперативной памяти микро-ЭВМ. Это позволяет эффективно проследивать, например, содержимое стека, значение различных счетчиков и т. д. Трассировка задается частичная или глобальная. При глобальной трассировке строки распечатываются после интерпретации каждой команды микро-ЭВМ на протяжении всей программы или на заданных участках программы. При частичной трассировке строки распечатываются после интерпретации команд микро-ЭВМ из заданного набора, например, после интерпретации команд перехода.

Команды разгрузки позволяют распечатывать содержимое областей памяти микро-ЭВМ в требуемых точках отлаживаемой программы. Таким образом, использование команд разгрузки и трассировки дает возможность быстро и эффективно выявлять несоответствия в работе программ. Следует отметить, что невыявление несоответствий даже на многочисленных, но произвольно выбранных тестовых примерах не является достаточным условием правильности программы. Доказательство правильности программы представляет собой сложную самостоятельную задачу.

При отладке небольших по объему программ, а также в ряде случаев и при отладке сравнительно больших программ с помощью команд разгрузки и трассировки можно определить и местонахождение ошибки, вызвавшей обнаруженное несоответствие в работе программы. Однако, в общем случае, для быстрого и эффективного обнаружения ошибок в программе по выявленным несоответствиям в ее работе служат команды событий. Сущность применения команд событий заключается в следующем. Ошибка, удаленная во времени и по месту расположения в программе от выявленного несоответствия, порождает то или иное событие, не предусмотренное правильной работой программы. Следовательно, поиск места нахождения ошибки сводится к выявлению некоторого события в программе, что легко формализуется. События можно разделить на элементарные, составные и условные.

К элементарным событиям, например, относятся: а) содержимое объекта больше заданного числа; б) содержимое объекта равно заданному числу; в) содержимое объекта меньше заданного числа; г) запись в объект; д) считывание из объекта; е) выполнение команды с заданным кодом операции. Под объектами подразумеваются ячейки памяти, аккумулятор, регистры и счетчик команд микропроцессора, регистры схем сопряжения и интерфейсов. Каждому элементарному событию можно поставить в соответствие логическую переменную, принимающую истинное значение, если событие имеет место, и ложное — в противоположном случае. Логическим функциям, построенным на основе указанных переменных, ставятся в соответствие составные события. Примером составных событий является обращение к объекту (дизъюнкция элементарных собы-

тий г и д), передача управления по адресу из заданных диапазонов адресов (дизъюнктивная нормальная форма от логических переменных, соответствующих элементарным событиям по пп. а, б и в, если объектом является счетчик команд) и т. д.

Условное событие — это упорядоченная последовательность элементарных и составных событий, происходящих во времени согласно заданному упорядочению. Примером условного события служит запись в некоторую ячейку памяти после выполнения команды по заданному адресу, многократное выполнение элементарного или составного события и т. д. Поскольку условному событию можно также поставить в соответствие логическую переменную, то события могут быть, в свою очередь, использованы при построении составных событий.

События, задаваемые в командах событий, именуются с помощью идентификаторов. Идентификаторы элементарных событий являются зарезервированными. При возникновении события, указанного в команде событий, выдается идентификатор события и строка трассировки. Кроме того, целесообразным является управление выполнением команд трассировки и разгрузки с помощью команд событий — задание или отказ от трассировки или разгрузки в зависимости от выполнения того или иного события.

Выявленные ошибки исправляются в тексте программы на машинном языке, затем из исправленной программы формируется объектный файл, записывается на ВН и цикл отладки повторяется.

Исходные программы отлаживаются с помощью эмулятора, обеспечивающего трансляцию исходных программ с автоматическим формированием объектного файла и его записью вместе с таблицами имен и меток на ВН (как правило, на МБ). Если в исходных программах синтаксических ошибок не обнаружено, управление автоматически передается на интерпретатор. При этом в командах трассировки, разгрузки и событий указывается не конкретные адреса ячеек оперативной памяти, а соответствующие им метки. Эффективность применения эмулятора объясняется тем, что он моделирует обычный цикл прохождения исходных программ на УЭВМ — трансляцию, с выдачей листинга, и печать транслированной программы с выдачей протокола от-

ладки. Однако, учитывая сравнительно низкое быстродействие кросс-ассемблеров и кросс-компиляторов, многократная перетрансляция при отладке особенно больших по объему исходных программ оказывается весьма дорогостоящей. Поэтому большие по объему исходные программы целесообразно отлаживать в два приема: вначале однократная трансляция с записью на ВН объектного файла и таблиц имен и меток, затем много-



Рис. 3. Структура резидентной СПО

кратная интерпретация объектной программы для выявления несоответствий и поиска ошибок. По отлаженным программам с помощью обслуживающей части кросс-ПО подготавливается требуемая документация.

Программы для микро-ЭВМ составляются на машинном языке или на языке ассемблера. Программы на машинном языке подготавливаются на перфоленте с помощью обслуживающей части кросс-ПО, сервисного блока монитора или резидентного макроассемблера. Считывание программы с перфоленты осуществляется монитором в режиме загрузки в память с ожиданием последующих команд или в режиме загрузки с последующей передачей управления на загруженную программу. Загруженная программа пользователя может обращаться к драйверам физических ВУ микро-ЭВМ.

Работа программы пользователя может быть прервана по любому из двух адресов, задаваемых с пульта. Два адреса задаются при необходимости останова программы после выполнения команды условного перехода. После останова управление может быть передано на любую команду программы пользователя.

Структура резидентной СПО изображена на рис. 3. Перед началом работы микро-ЭВМ с пульта задается соответствие имеющихся физических ВУ логическим ВУ. В исходном состоянии пульта является ЭПМ типа «Консул».

Программы для микро-ЭВМ составляются на машинном языке или на языке ассемблера. Программы на

Учитывая небольшой объем памяти, отведенной под монитор, блок отладки реализует только команды трассировки и разгрузки, причем в строку трассировки помещается лишь вектор состояния программы пользователя: содержимое счетчика команд, регистров, аккумулятора микропроцессора, а также код операции. Для выполнения трассировки организовано покомандное выполнение программы пользователя путем использования программного прерывания. Сущность организации такого режима выполнения пользовательской программы состоит в следующем.

До передачи управления на очередную команду программы пользователя монитор сохраняет следующую за очередной командой пользовательской программы. Если очередной командой является команда безусловного перехода, то сохраняется команда по адресу безусловного перехода. Если очередной командой является команда условного перехода, то сохраняется как следующая команда, так и команда по адресу перехода. На место сохраненных команд засылается команда прерывания. Затем вектор состояния монитора запоминается, а вектор состояния программы пользователя восстанавливается. По выполнении прерывания управление передается на специальную подпрограмму блока отладки, которая сохраняет вектор состояния пользовательской программы, восстанавливает соответствующие команды пользовательской программы и восстанавливает вектор состояния монитора. В зависимости от содержимого счетчика команд вектора состояния пользовательской программы, в случае глобальной трассировки, или кода выполненной команды, в случае частичной трассировки, монитор выдает на пульт строку трассировки.

Команды разгрузки реализуются после останова пользовательской программы по заданному адресу. По заданию команд разгрузки на пульт выдается содержимое требуемых областей памяти или регистров и аккумулятора микропроцессора.

Как отмечалось выше, команды разгрузки и трассировки не всегда позволяют быстро и эффективно обнаруживать местонахождение ошибки в программе. Для повышения эффективности отладки реализован режим передачи управления по некоторому адресу оперативной памяти после выполнения каждой команды пользовательской программы. По указанному адресу поль-

зователь может разместить свою программу для реализации отладки по событиям, применительно к конкретной ситуации отладки, дополнения строки трассировки содержимым требуемых ячеек оперативной памяти и т. д. По окончании такой программы управление должно быть возвращено на монитор.

Для исправления обнаруженных ошибок, оперативного изменения условий работы программы, а также для записи с пульта небольших программ используется команда модификации содержимого ячеек оперативной памяти, регистров и аккумулятора микропроцессора.

Отлаженная программа выдается на перфоленду для хранения, последующего ввода и эксплуатации, для подготовки требуемой документации с помощью обслуживающей части кросс-ПО.

Исходные программы на языке ассемблера подготавливаются с помощью обслуживающей части кросс-ПО или с помощью резидентного редактора. Для подготовки исходных программ с помощью резидентного редактора последний посредством монитора загружается с перфоленды в оперативную память. Затем с пульта рабочая область редактора заполняется текстом исходной программы с последующей выдачей содержимого рабочей области на печать и перфоленду. С учетом небольшого объема рабочей области указанный процесс повторяется многократно.

Для трансляции исходных программ с помощью монитора с перфоленды в оперативную память загружается макроассемблер. Ввод исходной программы макроассемблером осуществляется путем использования соответствующих блоков монитора. В связи со сравнительно небольшим объемом оперативной памяти микро-ЭВМ, исходная программа сначала не запоминается, а в последующем ее необходимо повторно вводить с перфоленды. При этом с пульта задается режим работы ассемблера на втором просмотре: выдача листинга исходной программы, выдача объектной программы на перфоленду или одновременная выдача листинга и объектной программы на перфоленду. Обнаруженные синтаксические ошибки в исходной программе исправляются посредством редактора. Для этого редактор загружается в оперативную память, исходная программа по частям считывается с перфоленды в рабочую область редактора, исправляется путем задания соответствующих

команд с пульта и выводится на перфоленду. Выданная на перфоленду объектная программа отлаживается с помощью монитора.

Достоинства и недостатки рассмотренных СПО. Учитывая, что кросс-ПО в силу мощных ресурсов оперативной и внешней памяти УЭВМ составляется на языках высокого уровня, разработка кросс-СПО менее трудоемка, чем разработка резидентной СПО. Как следствие больших возможностей УЭВМ кросс-СПО обладает большими возможностями отладки программ и подготовки требуемой документации. Кросс-СПО является более доступной, так как парк УЭВМ достаточно обширен, а приобретение комплекса программ проще и дешевле приобретения микро-ЭВМ с резидентной СПО. Разработка кросс-ПО, как правило, является первым этапом разработки ПО ЭВМ, чтобы с его помощью на последующих этапах отладить резидентное ПО.

В то же время затраты на эксплуатацию резидентной СПО значительно ниже, так как машинное время УЭВМ существенно дороже машинного времени микро-ЭВМ. Кроме того, резидентное СПО позволяет более эффективно отлаживать программы, предназначенные для управления объектами в реальном масштабе времени. Дальнейшие пути развития резидентной СПО связаны с усовершенствованием микро-ЭВМ, созданием малогабаритных внешних накопителей информации.

В заключение отметим, что рассмотренная кросс-СПО ориентирована на пакетный режим обработки на УЭВМ. Организация интерактивного режима не представляет принципиальных затруднений и вместе с тем дает дополнительные возможности для отладки программ микро-ЭВМ. Дальнейшие пути развития кросс-СПО заключаются в повышении эффективности и возможностей отладки программ, в развитии средств доказательства правильности программ.

УДК 621.374.32

Ю. И. ЩЕТИНИН, В. В. ВОРОБЬЕВА

МНОГОФУНКЦИОНАЛЬНОЕ ПЕРЕСЧЕТНОЕ УСТРОЙСТВО

Включение в микропроцессорный комплект схем, позволяющих осуществлять функции делителей частоты и формирователей импульсов, расширяет логические воз-